

How `genlasso.trendfilter.cv` Works in the 1D Benchmark

gflow metric-lowpass benchmark notes

May 16, 2026

1 What is `genlasso.trendfilter.cv`?

`genlasso.trendfilter.cv` is not an exported function from the `genlasso` package. It is the benchmark label used in our report for the following wrapper:

```
solution path: genlasso::trendfilter(y, pos = x, ord = 2),  
tuning:        genlasso::cv.trendfilter(fit, k = 5, mode = "lambda"),  
final fit:     coef(fit, lambda = cv$lambda.min)$beta.
```

Thus the actual algorithm is *quadratic trend filtering* with a five-fold cross-validated choice of the regularization parameter λ .

The name “lasso” can be confusing here. In high-dimensional regression, the ordinary lasso penalizes the absolute values of regression coefficients. Trend filtering instead penalizes the absolute values of *discrete derivatives* of the fitted curve. The lasso idea is the same—an ℓ_1 penalty creates sparsity—but the sparse objects are not selected covariates. They are selected locations where the fitted curve is allowed to change curvature.

2 The data and the fitted values

Assume the observations are ordered by position:

$$0 \leq x_1 < x_2 < \cdots < x_n \leq 1, \quad y_i = f(x_i) + \varepsilon_i.$$

Trend filtering estimates the vector of fitted values

$$\theta = (\theta_1, \dots, \theta_n)^T, \quad \theta_i \approx f(x_i).$$

It does not first choose knots and then fit a spline. Instead, it estimates the values θ_i directly, while penalizing changes in discrete derivatives.

3 The basic optimization problem

For polynomial order k , trend filtering solves

$$\hat{\theta}_\lambda = \arg \min_{\theta \in \mathbb{R}^n} \left\{ \frac{1}{2} \sum_{i=1}^n (y_i - \theta_i)^2 + \lambda \left\| D^{(k+1)} \theta \right\|_1 \right\}.$$

Here:

- The first term keeps the fitted values close to the noisy data.
- The second term penalizes roughness.
- $D^{(k+1)}\theta$ is a vector of discrete $(k+1)$ st derivatives.
- The ℓ_1 norm,

$$\left\| D^{(k+1)}\theta \right\|_1 = \sum_j \left| (D^{(k+1)}\theta)_j \right|,$$

encourages many of those discrete derivatives to be exactly zero.

In our benchmark, `ord` = 2, so $k = 2$. The fitted function is therefore locally piecewise quadratic, and the penalty is on third-order discrete differences:

$$\hat{\theta}_\lambda = \arg \min_{\theta \in \mathbb{R}^n} \left\{ \frac{1}{2} \sum_{i=1}^n (y_i - \theta_i)^2 + \lambda \sum_{j=1}^{n-3} \left| (D^{(3)}\theta)_j \right| \right\}.$$

4 What do the differences mean?

For equally spaced points, the first difference is

$$(D^{(1)}\theta)_i = \theta_{i+1} - \theta_i.$$

This measures local slope. The second difference is

$$(D^{(2)}\theta)_i = \theta_{i+2} - 2\theta_{i+1} + \theta_i.$$

This measures local curvature. The third difference is

$$(D^{(3)}\theta)_i = \theta_{i+3} - 3\theta_{i+2} + 3\theta_{i+1} - \theta_i,$$

up to an irrelevant sign convention. This measures changes in curvature.

For `ord` = 2, the ℓ_1 penalty tries to make most third differences zero. If the third difference is zero over a run of adjacent points, then the fitted values over that run lie on a quadratic polynomial. Where a third difference is nonzero, the curve can change quadratic piece. Those locations behave like adaptively chosen knots.

5 Why this is still lasso

The ordinary lasso has the form

$$\arg \min_b \left\{ \frac{1}{2} \|y - Xb\|_2^2 + \lambda \|b\|_1 \right\}.$$

It makes many entries of b exactly zero.

Trend filtering is a *generalized lasso*:

$$\arg \min_\theta \left\{ \frac{1}{2} \|y - \theta\|_2^2 + \lambda \|D\theta\|_1 \right\}.$$

Now the lasso penalty is applied after multiplying by a structured matrix D . In trend filtering, $D = D^{(k+1)}$, so the sparse quantities are discrete derivatives. Sparsity in derivatives means the fitted curve is simple on long intervals and only changes shape at a limited number of data-adaptive locations.

6 Unequally spaced x -values

Our benchmark samples x_i values randomly in $[0, 1]$, so the gaps $x_{i+1} - x_i$ are not constant. The `genlasso` implementation supports this through the `pos` argument. Internally, it replaces ordinary finite differences by weighted finite differences, equivalent in spirit to divided differences.

For first differences, the natural scaling is

$$\frac{\theta_{i+1} - \theta_i}{x_{i+1} - x_i}.$$

For higher orders, the package recursively builds a difference operator using weights of the form

$$\frac{m}{x_{i+m} - x_i},$$

at derivative level m . This adjusts the derivative penalty so that a large gap in x is not treated the same way as a small gap.

The package documentation warns that arbitrary positions can be numerically more delicate than equally spaced positions, especially for high polynomial orders. Our wrapper uses $k = 2$, which is moderate: it allows curved fits but does not go into the higher-order range where instability is more likely.

7 The role of λ

The tuning parameter λ controls the tradeoff between fidelity and simplicity:

$$\frac{1}{2}\|y - \theta\|_2^2 \quad \text{versus} \quad \lambda\|D^{(3)}\theta\|_1.$$

λ	Effect on θ	Interpretation
very large	many third differences equal zero	very few knots, smooth simple curve
moderate	some nonzero third differences	adaptive piecewise quadratic curve
near zero	many nonzero third differences	curve follows data more closely

Large λ says: “Only bend the curve when the evidence is strong.” Small λ says: “Fit the observations more closely, even if that requires many local bends.”

8 Visual picture

9 How cross-validation chooses λ

`genlasso::trendfilter()` computes a solution path: a sequence of λ values at which the fitted curve changes. Then `genlasso::cv.trendfilter()` chooses among these candidate values.

For trend filtering, the package uses deterministic ordered folds rather than random folds. Roughly, every k th interior point is assigned to the same fold, while the first and last points remain in the training set. For a held out point, the prediction is computed from neighboring fitted values. The selected value

$$\lambda_{\min} = \arg \min_{\lambda} \text{CV}(\lambda)$$

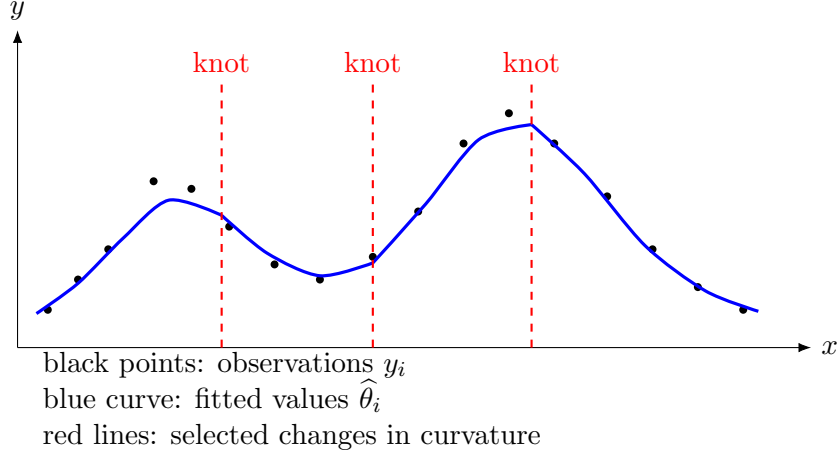


Figure 1: Quadratic trend filtering estimates fitted values at the observed positions. Most third differences are zero, so the curve behaves like a quadratic polynomial between selected knots.

is the one with the smallest cross-validated prediction error.

In our benchmark, we use:

$$k_{\text{fold}} = 5, \quad \text{mode} = \text{"lambda"}, \quad \lambda = \lambda_{\min}.$$

Thus the method is allowed to adapt its smoothness separately for each simulated dataset.

10 Why it did so well in the 1D benchmark

The benchmark truth is a smooth function on a one-dimensional interval, built from mixtures of Gaussian bumps. Trend filtering is a strong method for exactly this kind of problem because:

1. It uses the true ordering of the data on the line.
2. It adapts knot locations to the observed signal rather than placing knots uniformly.
3. The ℓ_1 penalty allows local flexibility where the signal bends sharply, while keeping flatter regions simple.
4. Cross-validation chooses the roughness level from the data.

This also explains why the result is not a contradiction of the usual story that lasso is useful in high-dimensional settings. The lasso mechanism is more general than variable selection. Here it is being used for sparse changes in the curve's derivatives.

11 Relationship to the graph low-pass smoother

The graph low-pass smoother estimates f by suppressing high-frequency components of a graph Laplacian. On the path graph, this is also a kind of 1D regularization, but the penalty is quadratic and spectral:

$$\sum_i (y_i - f_i)^2 + \eta f^T L f$$

or, after spectral filtering, an equivalent shrinkage of high-frequency eigenvectors.

Trend filtering instead uses an ℓ_1 penalty on higher-order differences:

$$\sum_i (y_i - \theta_i)^2 + \lambda \|D^{(3)}\theta\|_1.$$

The distinction matters. Quadratic penalties tend to smooth everywhere. ℓ_1 derivative penalties tend to smooth most places while allowing localized bends. That localized adaptivity is a likely reason trend filtering was so strong in the path-graph 1D benchmark.

12 References

- Tibshirani, R. J. and Taylor, J. (2011). The solution path of the generalized lasso. *Annals of Statistics*, 39(3), 1335–1371.
- Tibshirani, R. J. (2014). Adaptive piecewise polynomial estimation via trend filtering. *Annals of Statistics*, 42(1), 285–323.
- Arnold, T. B. and Tibshirani, R. J. (2014). Efficient implementations of the generalized lasso dual path algorithm. arXiv:1405.3222.
- Kim, S.-J., Koh, K., Boyd, S. and Gorinevsky, D. (2009). ℓ_1 trend filtering. *SIAM Review*, 51(2), 339–360.